

Formation Développement Web

Semaine 2 – Séance 2

Layout & Effects : Flexbox, Grid, Responsive Design et transitions

Mars 2026

Objectifs de la séance

0

À la fin de cette séance, vous serez capables de :

- comprendre les principes de la **mise en page moderne** en CSS ;
- utiliser **Flexbox** pour aligner et répartir des éléments ;
- utiliser **CSS Grid** pour construire des grilles simples ;
- adapter une page à différentes tailles d'écran avec les **media queries** ;
- ajouter des effets visuels simples avec **:hover** et **transition**.

Plan de la séance

0

Partie 1 – Mise en page moderne

- Pourquoi penser au layout ?
- Flexbox
- CSS Grid

Partie 2 – Responsive Design

- Pourquoi une page doit s'adapter ?
- Introduction aux media queries

Partie 3 – Interactivité visuelle

- pseudo-classe :hover
- propriété transition
- mise en pratique sur un mini layout

Table of Contents

1 Mise en page moderne

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Pourquoi parler de mise en page ?

1 Mise en page moderne

Jusqu'ici, nous avons appris à :

- changer les couleurs ;
- modifier les polices ;
- ajouter des bordures et des espacements.

Mais une vraie page web doit aussi savoir :

- placer les éléments correctement ;
- organiser le contenu ;
- s'adapter à la taille de l'écran ;
- rester lisible et agréable.

Idée importante

Le CSS ne sert pas seulement à décorer une page, il sert aussi à **l'organiser**.

Deux outils importants pour le layout

1 Mise en page moderne

En CSS moderne, deux outils sont très utilisés :

Flexbox

Très pratique pour aligner des éléments dans **une seule direction** :

- en ligne ;
- ou en colonne.

CSS Grid

Très pratique pour construire une **grille** avec lignes et colonnes.

Table of Contents

2 Flexbox

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Qu'est-ce que Flexbox ?

2 Flexbox

Flexbox est un système de mise en page CSS.

Il permet de :

- aligner facilement des éléments ;
- gérer les espacements ;
- répartir les blocs dans une ligne ou une colonne ;
- centrer du contenu plus simplement.

À retenir

Flexbox est surtout adapté aux mises en page **unidimensionnelles**.

Pour utiliser Flexbox, on applique `display: flex;` à un conteneur.

```
.container {  
  display: flex;  
}
```

- l'élément `.container` devient un conteneur flexible ;
- ses enfants sont disposés automatiquement avec les règles Flexbox.

Axe principal et axe secondaire

2 Flexbox

Avec Flexbox, il faut comprendre deux axes :

Axe principal

C'est la direction dans laquelle les éléments sont placés.

Axe secondaire

C'est l'axe perpendiculaire à l'axe principal.

Par défaut :

- l'axe principal est horizontal ;
- l'axe secondaire est vertical.

Changer la direction avec flex-direction

2 Flexbox

```
.container {  
  display: flex;  
  flex-direction: row;  
}
```

Valeurs importantes :

- row : éléments en ligne;
- column : éléments en colonne.

Aligner les éléments avec justify-content

2 Flexbox

justify-content agit sur l'axe principal.

```
.container {  
  display: flex;  
  justify-content: center;  
}
```

Quelques valeurs utiles :

- flex-start
- center
- space-between
- space-around
- space-evenly

Aligner les éléments avec align-items

2 Flexbox

align-items agit sur l'axe secondaire.

```
.container {  
  display: flex;  
  align-items: center;  
}
```

Cette propriété permet par exemple de :

- aligner verticalement des éléments ;
- centrer des blocs dans un menu ;
- harmoniser des cartes de même hauteur.

Ajouter de l'espace avec gap

2 Flexbox

```
.container {  
  display: flex;  
  gap: 20px;  
}
```

- gap ajoute de l'espace entre les éléments ;
- c'est plus simple et plus propre que de mettre des marges partout.

Exemple : une barre de navigation avec Flexbox

2 Flexbox

```
nav {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
}
```

Effet obtenu :

- le logo peut être placé à gauche ;
- les liens peuvent être placés à droite ;
- l'ensemble reste propre et lisible.

Exemple : centrer parfaitement un bloc

2 Flexbox

```
.hero {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```

Résultat

Le contenu est centré horizontalement et verticalement.

Quand utiliser Flexbox ?

2 Flexbox

Flexbox est très utile pour :

- une navigation ;
- une rangée de cartes ;
- un bloc à centrer ;
- une disposition en colonne ;
- de petits layouts simples.

À retenir

Flexbox fonctionne très bien quand on veut organiser les éléments dans **une direction principale**.

Table of Contents

3 CSS Grid

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Qu'est-ce que CSS Grid ?

3 CSS Grid

CSS Grid est un système de mise en page basé sur une grille.

Il permet de :

- créer plusieurs colonnes ;
- créer plusieurs lignes ;
- répartir des blocs dans une structure plus précise ;
- construire des layouts plus complexes.

À retenir

Grid est particulièrement adapté aux mises en page **bidimensionnelles**.

Activer Grid

3 CSS Grid

```
.container {  
  display: grid;  
}
```

Un conteneur Grid permet ensuite de définir :

- le nombre de colonnes ;
- le nombre de lignes ;
- l'espace entre les cellules.

Créer des colonnes avec grid-template-columns

3 CSS Grid

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
}
```

Cela signifie :

- 2 colonnes ;
- chaque colonne prend la même largeur.

fr signifie **fraction de l'espace disponible**.

Exemple avec trois colonnes

3 CSS Grid

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}
```

- les éléments sont organisés sur 3 colonnes ;
- gap ajoute l'espace entre les blocs.

Exemple concret : grille de cartes

3 CSS Grid

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 20px;  
}
```

On peut utiliser cela pour :

- une galerie ;
- une liste de projets ;
- des cartes de compétences ;
- des sections de dashboard simples.

Flexbox ou Grid ?

3 CSS Grid

Flexbox

Idéal pour :

- aligner ;
- centrer ;
- gérer une ligne ou une colonne.

Grid

Idéal pour :

- construire une grille ;
- gérer lignes et colonnes ;
- créer une structure plus large.

Table of Contents

4 Responsive Design

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ **Responsive Design**
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Pourquoi parler de Responsive Design ?

4 Responsive Design

Aujourd'hui, un site web peut être consulté sur :

- un téléphone ;
- une tablette ;
- un ordinateur portable ;
- un grand écran.

Problème

Une page qui est jolie sur ordinateur peut être difficile à lire sur téléphone.

Solution

Le **Responsive Design** permet d'adapter l'affichage à la taille de l'écran.

Objectif du responsive

4 Responsive Design

Une page responsive doit :

- rester lisible ;
- éviter les débordements ;
- réorganiser les éléments si nécessaire ;
- offrir une bonne expérience sur mobile.

Exemple :

- sur ordinateur : 3 cartes en ligne ;
- sur téléphone : les cartes passent en colonne.

Introduction aux media queries

4 Responsive Design

Une **media query** permet d'appliquer du CSS seulement dans certaines conditions.

```
@media (max-width: 768px) {  
  body {  
    background-color: #ffffff;  
  }  
}
```

- max-width: 768px signifie : écran de largeur inférieure ou égale à 768 pixels ;
- le CSS à l'intérieur ne s'applique que dans ce cas.

Exemple responsive avec Flexbox

4 Responsive Design

```
.nav-links {  
  display: flex;  
  gap: 20px;  
}  
  
@media (max-width: 768px) {  
  .nav-links {  
    flex-direction: column;  
  }  
}
```

Résultat :

- sur grand écran, les liens sont en ligne ;
- sur petit écran, ils passent en colonne.

Exemple responsive avec Grid

4 Responsive Design

```
.cards {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr;  
  gap: 20px;  
}  
  
@media (max-width: 768px) {  
  .cards {  
    grid-template-columns: 1fr;  
  }  
}
```

Résultat

La grille passe de 3 colonnes à 1 seule colonne sur petit écran.

Bonnes pratiques responsive

4 Responsive Design

- tester la page sur plusieurs tailles d'écran ;
- éviter les largeurs fixes trop grandes ;
- utiliser des espacements raisonnables ;
- simplifier le layout sur mobile ;
- penser d'abord à la lisibilité.

Table of Contents

5 Interactivité visuelle

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ **Interactivité visuelle**
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Pourquoi ajouter des effets visuels ?

5 Interactivité visuelle

Les effets visuels simples permettent de :

- rendre l'interface plus vivante ;
- améliorer le retour visuel ;
- attirer l'attention sur un bouton ou un lien ;
- donner une impression plus moderne.

Attention

Les effets doivent rester simples, utiles et lisibles.

La pseudo-classe :hover

5 Interactivité visuelle

:hover permet de changer l'apparence d'un élément quand la souris passe dessus.

```
button:hover {  
    background-color: darkblue;  
}
```

On l'utilise souvent pour :

- les boutons ;
- les liens ;
- les cartes ;
- les images.

Exemple simple avec un lien

5 Interactivité visuelle

```
a {  
  color: black;  
}  
  
a:hover {  
  color: blue;  
}
```

Effet obtenu

Le lien change de couleur au survol.

La propriété transition

5 Interactivité visuelle

transition permet d'adoucir un changement visuel.

```
button {  
    background-color: steelblue;  
    transition: 0.3s;  
}  
  
button:hover {  
    background-color: darkblue;  
}
```

Sans transition, le changement est immédiat. Avec transition, le changement devient plus fluide.

Transition sur plusieurs propriétés

5 Interactivité visuelle

```
.card {  
  transition: transform 0.3s, box-shadow 0.3s;  
}  
  
.card:hover {  
  transform: translateY(-5px);  
}
```

Cela permet par exemple :

- de faire monter légèrement une carte ;
- de rendre l'interface plus dynamique ;
- d'améliorer la perception visuelle.

Effets visuels à utiliser avec modération

5 Interactivité visuelle

Il faut éviter :

- trop d'animations ;
- des changements trop brusques ;
- des couleurs agressives ;
- des effets qui gênent la lecture.

Bonne pratique

Un bon effet visuel doit aider l'utilisateur, pas le distraire.

Table of Contents

6 Exemple complet

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Exemple CSS complet

6 Exemple complet

```
body {  
  font-family: Arial, sans-serif;  
  margin: 0;  
  background-color: #f5f5f5;  
}  
  
nav {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 20px;  
  background-color: white;  
}  
  
.cards {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
  gap: 20px;  
  padding: 20px;  
}
```

```
card {
```


Table of Contents

7 Mise en pratique

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Code HTML de départ

7 Mise en pratique

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Layout & Effects</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <nav>
    <h1>Mon Site</h1>
    <div class="nav-links">
      <a href="#">Accueil</a>
      <a href="#">Projets</a>
      <a href="#">Contact</a>
    </div>
  </nav>

  <section class="cards">
    <article class="card">
      <h2>Flexbox</h2>
      <p>Alignement simple des éléments.</p>
    </article>

    <article class="card">
      <h2>Grid</h2>
      <p>Organisation en colonnes et lignes.</p>
    </article>
  </section>
</body>
</html>
```

Exercice pratique

7 Mise en pratique

À partir du code HTML fourni :

1. organiser la navigation avec Flexbox ;
2. créer une grille de cartes avec CSS Grid ;
3. ajouter un gap entre les cartes ;
4. ajouter un effet `:hover` sur les cartes ;
5. ajouter une transition fluide ;
6. rendre la page responsive avec une media query.

Résultat attendu

7 Mise en pratique

À la fin de l'exercice :

- la navigation doit être bien alignée ;
- les cartes doivent être disposées dans une grille ;
- un effet visuel doit apparaître au survol ;
- l'affichage doit s'adapter sur petit écran ;
- la page doit être propre, lisible et structurée.

Important

L'objectif n'est pas de produire un design parfait, mais de bien comprendre la logique du layout moderne.

Erreurs fréquentes à éviter

7 Mise en pratique

- oublier `display: flex` ou `display: grid`;
- confondre `justify-content` et `align-items`;
- oublier `gap` et surcharger la page avec des marges;
- écrire une media query sans tester le résultat;
- utiliser trop d'effets visuels;
- oublier `transition` quand on veut un effet fluide.

Récapitulatif de la séance

7 Mise en pratique

Aujourd'hui, nous avons appris à :

- organiser une page avec Flexbox ;
- créer une grille simple avec CSS Grid ;
- adapter une page à l'écran avec les media queries ;
- ajouter un effet `:hover` ;
- rendre un changement visuel plus fluide avec `transition`.

Table of Contents

8 Devoirs

- ▶ Mise en page moderne
- ▶ Flexbox
- ▶ CSS Grid
- ▶ Responsive Design
- ▶ Interactivité visuelle
- ▶ Exemple complet
- ▶ Mise en pratique
- ▶ Devoirs

Devoirs

8 Devoirs

À partir du mini projet de la séance :

- améliorer la navigation avec Flexbox ;
- créer une grille de 3 ou 4 cartes ;
- ajouter un effet :hover sur les cartes ;
- ajouter une transition propre ;
- faire en sorte que les cartes passent en une seule colonne sur mobile ;
- tester le rendu sur ordinateur et téléphone.

Consignes

Le code CSS doit rester propre, indenté et bien commenté si nécessaire.

Formation Développement Web

*Thank you for listening !
Any questions ?*